

Actually, it is possible to give the decoder a random input and generate an output similar to the input. However, it is necessary to continue training the NN in order to make it work. This is a generic example of generative AI.

COVERED IN CH. 14.1 OF

DEEP LEARNING BY GOODFELLOW, BENGIO, COURVILLE

N.B.: if linear functions are applied to  $f$  and  $g$ , they are basically equal to  $U$  and  $W$ . So the result will be a convex function where it is possible to find the local minimum or, in some cases, output could be the input itself. || (however, it is way more complicated than US P2A)

## ENSEMBLE METHODS

This topic is about combining some methods previously introduced to implement a more powerful approach to learning.

To start with, let's formalize the idea of WEAK LEARNER

### $\gamma$ -WEAK LEARNABILITY

A learning algorithm  $A$  is called a  $\gamma$ -WEAK LEARNER for a class if there exists a function  $m_H: (0, 1) \rightarrow \mathbb{N}$  s.t. for every  $\delta \in (0, 1)$ , for every distribution  $D$  over  $X$ , and for every labeling function  $f: X \rightarrow \{-1, +1\}$  if the reliability hypothesis holds (with  $D, H, f$ ) then when running  $A$  on  $m \geq m_H(\delta)$  i.i.d. examples generated by  $D$  and labelled by  $f$ ,  $A$  returns an hypothesis  $h$  s.t.  $L(D, f) \leq \frac{1}{2} - \gamma$ , with probability  $p \geq \delta$  || it says that it is not good predictor but it is better than a random guess

So, the idea is that a  $\gamma$ -WEAK LEARNER performs slightly better than a random guess on a classification problem.  $m \geq m_H(\delta)$  means that the # of samples must be sufficiently large in order to compare the performance of  $A$  to a random guess classifier. To formalize:

- $\delta$ : confidence  
 $\gamma$ : advantage w.r.t. random classifier
- $L(D, f) \leq \frac{1}{2} - \gamma$  with confidence  $\delta$   
 while training  $A$  on at least  $m_H(\delta)$  samples

This definition helps in recognising good learners.

Our goal is to combine some  $\gamma$ -WEAK learners in order to obtain a  $\gamma$ -STRONG learner.

What we will see is an example of BOOSTING a set of algorithms.

(BOOSTING) ADA BOOST (Adaptive Boosting)

$$S = \{(x_1, y_1), \dots, (x_m, y_m)\}, y_i = f(x_i)$$

ITERATIVE ALGORITHM

At iteration  $t$ :

1. Produce distribution  $\Delta^{(t)}$  over  $S$ .  
distribution defined by the algorithm

Initially  $\Delta^0 = \langle \frac{1}{m}, \dots, \frac{1}{m} \rangle$  (uniform distribution over  $S$ )

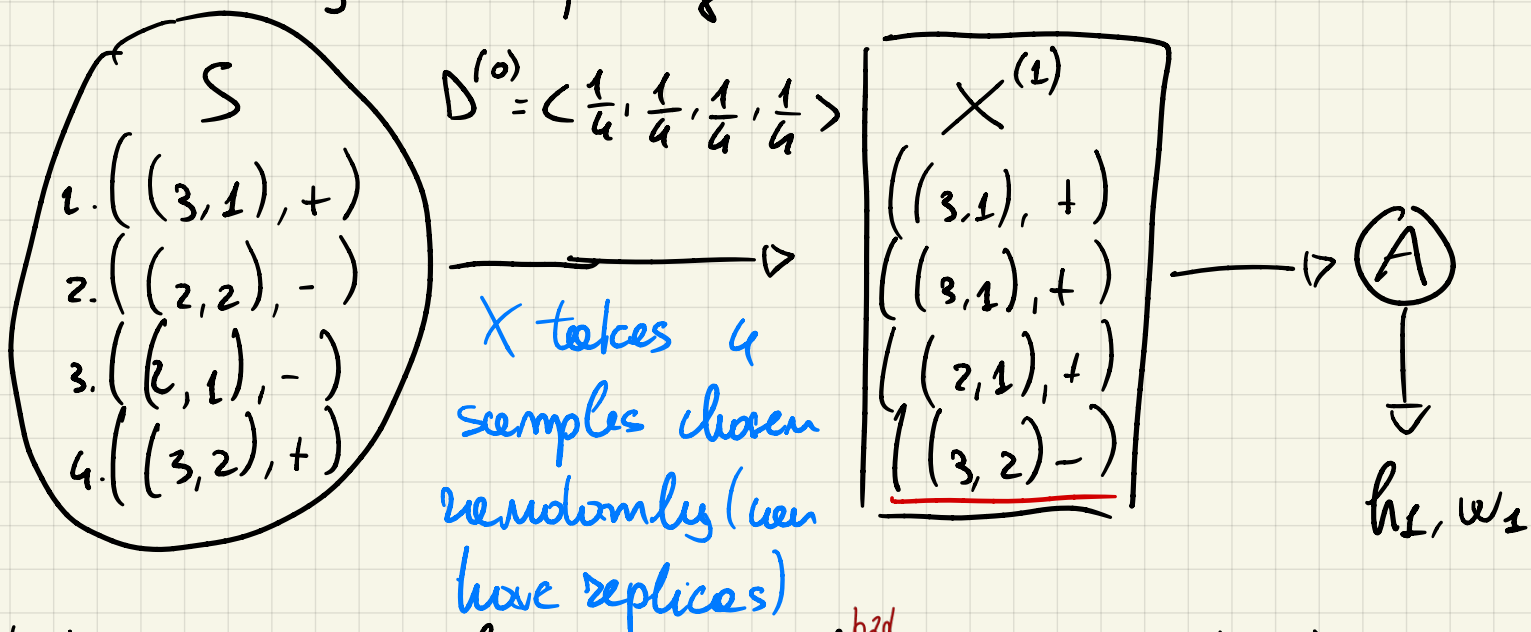
2. Train  $\widehat{A}(S, \Delta^{(t)}) \mapsto h_t$   
can be linear regression  
 $\epsilon_t = L_{\Delta^{(t)}}(h_t) = \sum_{i=1}^m \Delta_i^{(t)} \mathbb{1}[h_t(x_i) \neq y_i] \leq \frac{1}{2} - \gamma$   
 $\mathbb{1}[\varphi] = \begin{cases} 1, & \text{if } \varphi \text{ true} \\ 0, & \text{otherwise} \end{cases}$

Assign to  $h_t$  a weight  $w_t = \frac{1}{2} \log \left( \frac{1}{\epsilon_t} - 1 \right)$

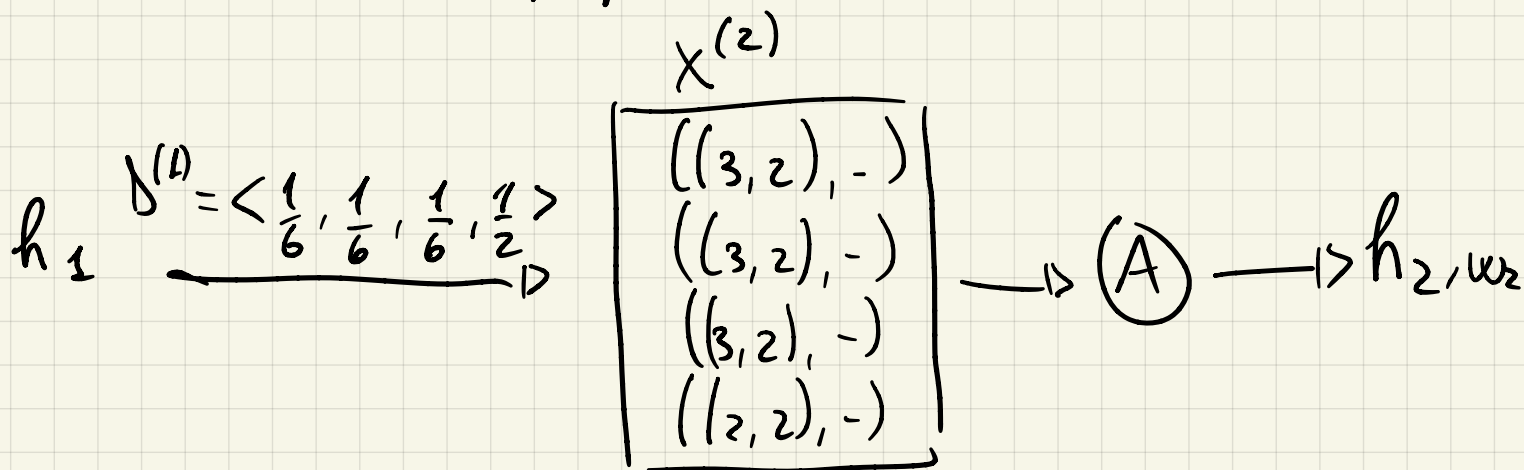
3. Update distribution s.t. misclassified samples by  $h_t$  get higher probability of being picked in the next distribution

### EXAMPLE

We start by 4 samples given  $S$ :



Let's say that  $h_1$  performs <sup>bad</sup> on  $((3,2), -)$ :



So,  $h_2$  will give much more importance on classifying  $((3,2), -)$  correctly. Notice also that  $w_i$  is always inversely proportional to  $\epsilon_i$  in order to apply this method.

At iteration  $t$ :

$$\left. \begin{matrix} h_1, \epsilon_1, w_1 \\ \vdots \\ h_t, \epsilon_t, w_t \end{matrix} \right\} h_S(x) = \text{sign} \left( \sum_{i=1}^T w_i h_i(x) \right)$$

$\rightarrow$  USED TO RETURN +1 OR -1

# AdaBoost

input:

training set  $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

weak learner WL

number of rounds  $T$

initialize  $\mathbf{D}^{(1)} = (\frac{1}{m}, \dots, \frac{1}{m})$ .

for  $t = 1, \dots, T$ :

invoke weak learner  $h_t = \text{WL}(\mathbf{D}^{(t)}, S)$

compute  $\epsilon_t = \sum_{i=1}^m D_i^{(t)} \mathbb{1}_{[y_i \neq h_t(\mathbf{x}_i)]}$

let  $w_t = \frac{1}{2} \log \left( \frac{1}{\epsilon_t} - 1 \right)$

update  $D_i^{(t+1)} = \frac{D_i^{(t)} \exp(-w_t y_i h_t(\mathbf{x}_i))}{\sum_{j=1}^m D_j^{(t)} \exp(-w_t y_j h_t(\mathbf{x}_j))}$  for all  $i = 1, \dots, m$

output the hypothesis  $h_s(\mathbf{x}) = \text{sign} \left( \sum_{t=1}^T w_t h_t(\mathbf{x}) \right)$ .

$t$ : initially  $\mathbf{D}^{(1)} = \langle D_1^{(1)}, D_2^{(1)}, \dots, D_m^{(1)} \rangle$

$$D_i^{(t+1)} = \frac{D_i^{(t)} \cdot e^{-w_t y_i h_t(\mathbf{x}_i)}}{\sum_{j=1}^m D_j^{(t)} e^{-w_t y_j h_t(\mathbf{x}_j)}} \quad \forall i = 1, \dots, m$$

if  $\text{sign}(y_i) \neq h_t(\mathbf{x}_i) \Rightarrow \text{sign}(y_i h_t(\mathbf{x}_i)) = -1$

SIMILAR  
TO A  
SOFTMAX  
FUNCTION

NORMALIZATION FACTOR

Actually, the  $D_i^{(t+1)}$  update can be sum up this way:  
- if  $h_t(\mathbf{x}_i)$  is misclassified ( $\text{sign}(y_i h_t(\mathbf{x}_i)) = -1$ ) then this sample,  $\mathbf{x}_i$ , will have higher probability of being picked into the next round.

To prove that ADABOOST is functioning, we have a theorem:

## THEOREM

If at every iteration  $t$  the WL (weak learner) returns a hypothesis s.t.  $\epsilon_t < \frac{1}{2} - \gamma$ , then the training error of the final hypothesis  $h_s$  is:



$$L_S(h_S) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}_{[h_S(x_i) \neq y_i]} e^{-2\gamma^2 T}$$

In a few words,  $h_S$  has a decreasing error proportional to the number of rounds  $T$ .

COMPUTATIONAL COST

### AdaBoost

input:

training set  $S = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

weak learner WL

number of rounds  $T$

initialize  $\mathbf{D}^{(1)} = (\frac{1}{m}, \dots, \frac{1}{m})$ .

for  $t = 1, \dots, T$ :

invoke weak learner  $h_t = \text{WL}(\mathbf{D}^{(t)}, S)$   $\hookrightarrow c_{WL}(m)$

compute  $\epsilon_t = \sum_{i=1}^m D_i^{(t)} \mathbb{1}_{[y_i \neq h_t(\mathbf{x}_i)]}$   $\hookrightarrow O(m)$

let  $w_t = \frac{1}{2} \log \left( \frac{1}{\epsilon_t} - 1 \right)$

update  $D_i^{(t+1)} = \frac{D_i^{(t)} \exp(-w_t y_i h_t(\mathbf{x}_i))}{\sum_{j=1}^m D_j^{(t)} \exp(-w_t y_j h_t(\mathbf{x}_j))}$   $\hookrightarrow O(m)$  for all  $i = 1, \dots, m$

output the hypothesis  $h_S(\mathbf{x}) = \text{sign} \left( \sum_{t=1}^T w_t h_t(\mathbf{x}) \right)$ .

COST:  $O(t \cdot m) \cdot c_{WL}(m) \sim O(t \cdot m)$

EFFICIENCY  
OF THE WL  $\parallel$  weak learner

Note that  $h_S(\mathbf{x}) = \text{sign} \left( \sum_{i=1}^T w_i h_i(\mathbf{x}) \right)$  works on classification.  
In a regression problem,  $h_S(\mathbf{x})$  must be adapted:

$$h_S(\mathbf{x}) = \sum_{i=1}^T w_i h_i(\mathbf{x})$$

$\hookrightarrow$  error could be the squared distance, for example

That is, applying the weighted sum of the hypotheses.

In general, ADABOOST can be applied to many algorithms.  
There is also a simpler approach, which is BAGGING

## BAGGING

Given a dataset  $S$ :

1. Generate  $m$  datasets  $X_1, \dots, X_m$  by random sampling  $S$  with replacement (can extract replicas of an element)
2. Train a model  $h_i$  <sup>or model (it is not mandatory a single model, and also can be different every iteration)</sup> (doesn't have to be a WL) on each dataset  $D_i$  (N.B.: for every dataset it is possible to use a different model)
3. Make predictions on new instances using a combination scheme:

<sup>or way to combine the output</sup>

$$y_S(x) = \frac{1}{M} \sum_{i=1}^m h_i(x) \quad (\text{REGRESSION})$$

$$y_S(x) = \operatorname{argmax}_c \sum_{i=1}^m \frac{1}{m} \mathbb{1}_{[h_m(x)=c]} \quad (\text{CLASSIFICATION})$$

In a few words, each model makes its own prediction on its dataset. Then, at the end of the day, every prediction is compared. The most frequent output will be the final result ( $y_S(x)$ )

N.B.: every prediction on a specific dataset is independent by predictions on other datasets (avoids bias)

This is a more powerful way of training a single model. It also mitigates overfitting.